

SAVI

Sistema de Acesso Virtual a Informação Clínica

Documentación del proceso de construcción y despliegue

Registro completo de lo realizado desde la creación del proyecto hasta la demo publicada: herramientas utilizadas, puntos críticos encontrados en cada estación del proceso, y sugerencias de mejora para las próximas iteraciones.

Fecha del informe: 9 de septiembre de 2026

Repositorio: pulseiraSAVI/Pulseira-Savi — demo publicada en <https://pulseirasavi.github.io/Pulseira-Savi/>

1. Resumen ejecutivo

Se construyó y publicó una **demo interactiva y funcional** de la primera iteración de SAVI: los tres flujos de la Fase 1 (profesional clínico, familia, equipa administradora), con el ecrã de Nivel 1 fiel al mockup de referencia, 2FA simulado, timeout de sesión real, y empaquetada como PWA instalable. Está publicada en GitHub Pages y es navegable por cualquier persona del equipo.

No es una aplicación en producción: no hay backend real (Firestore, Cloudflare Worker, Firebase Auth), los datos viven en localStorage del navegador, y el propio CLAUDE.md del proyecto exige EIPD/DPIA formal, DPAs con subprocesadores y clarificar el responsable del tratamiento antes de incluir cualquier paciente piloto real — pendientes que no dependen de código.

El proceso de construcción en sí avanzó sin sobresaltos; el **despliegue en GitHub** fue donde se concentraron casi todos los puntos de fricción — detallados en las secciones 2 y 3, junto con sugerencias concretas para evitarlos en la próxima iteración.

2. Cronología del proceso

Fase A — Especificación

- Lectura de CLAUDE.md y los tres documentos fuente (Resumo do Projeto, Arquitetura Técnica, Design App Papel) más el mockup HTML de referencia del ecrã de Nivel 1.
- Resultado: modelo de datos (schema SQL → colecciones Firestore), roles y permisos, flujo de scan/break-glass, y reglas no negociables confirmadas antes de escribir ninguna línea de código.

Fase B — Construcción de la demo

- Construcción del frontend (vanilla JS, sin frameworks) y del scaffolding de backend (Firestore Security Rules, esqueleto de Cloudflare Worker) mediante un subagente — que se interrumpió a mitad de camino por un error de conexión y tuvo que reanudarse con un segundo subagente al que se le dio contexto explícito de lo ya construido.
- Empaquetado como PWA: manifest, service worker con caché versionado, e iconos generados con ImageMagick a partir de un SVG propio.
- Ajustes de diseño responsive en varias vueltas sucesivas, a medida que se acotaba qué se quería ver en escritorio frente a móvil (de una moldura de teléfono decorativa, a un tope de ancho tipo portátil de 15", a pantalla completa sin tope en ningún tamaño).

Fase C — Publicación en GitHub Pages

- Conexión a la carpeta de proyecto del usuario y primer intento de preparar un repositorio git local — bloqueado parcialmente porque esa carpeta compartida no permite borrar ni renombrar archivos ya creados por otros procesos.
- El usuario ofreció un token de acceso personal de GitHub para que se usara directamente — rechazado por política (los tokens/API keys nunca se manejan, sin excepción), lo que generó confusión inicial al compararlo con la experiencia en otro proyecto con Claude Code.

- Subida del proyecto por la interfaz web de GitHub (arrastrar y soltar), lo que omitió en silencio los archivos y carpetas ocultos (.github, .nojekyll) — el sitio publicado terminó mostrando el README renderizado por Jekyll en vez de la app real, sin ningún mensaje de error.
- Intento de resolverlo por terminal con git — bloqueado por una credencial de otra cuenta de GitHub cacheada en el llavero de macOS, que dio un error 403 de permisos.
- Se decidió cambiar de estrategia a "Deploy from a branch", lo que requirió renombrar la carpeta de la app de frontend/ a docs/ (única carpeta que ese modo admite además de la raíz) y volver a subir los archivos, esta vez incluyendo los ocultos mediante "Add file → Create new file" con la ruta escrita a mano.

Fase D — Validación

- Verificación visual final: la URL pública carga el login real de la app (no el README), y el navegador ofrece instalarla como PWA — confirmando que el despliegue quedó correcto.

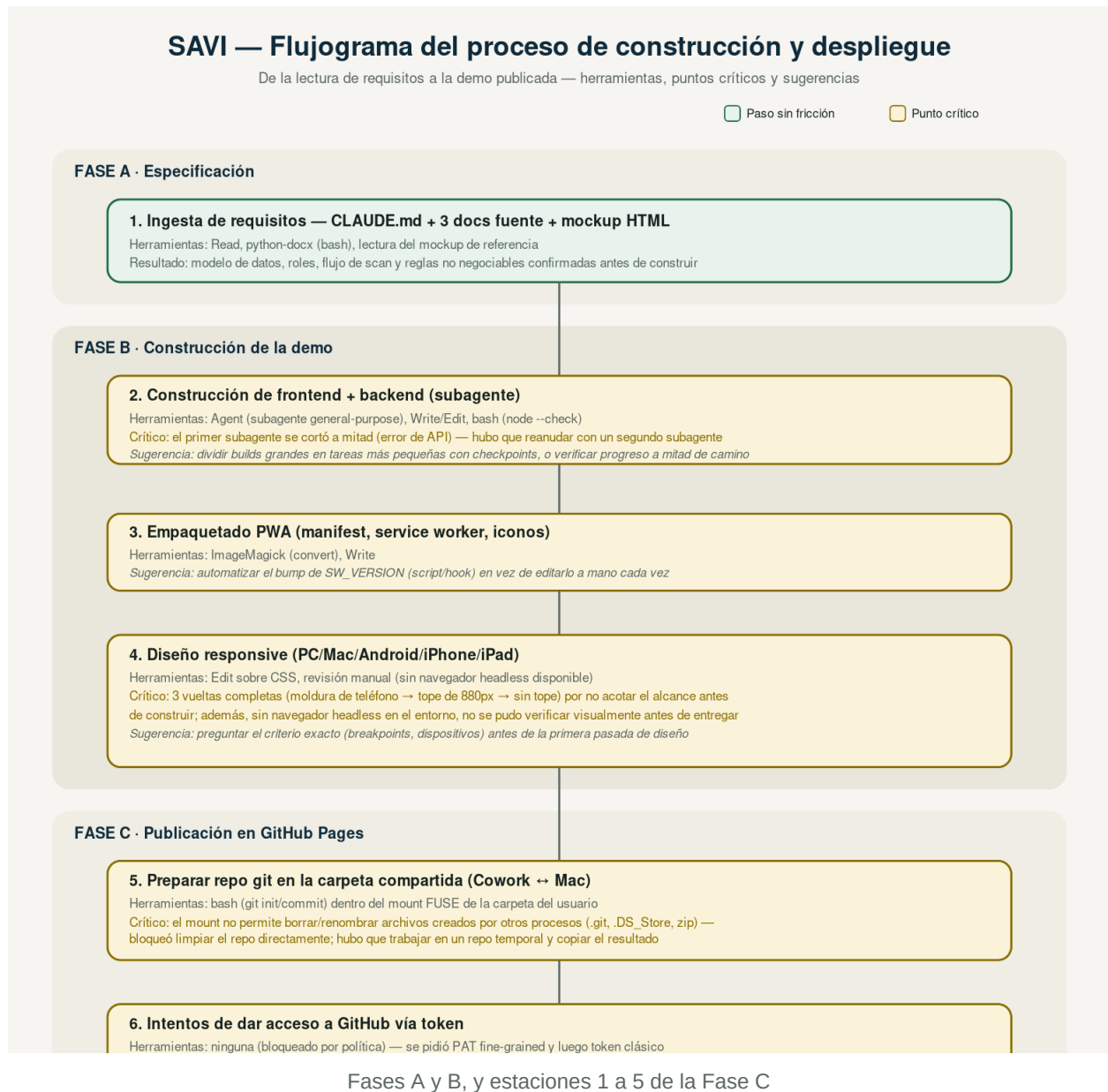
3. Herramientas y puntos críticos por estación

Cada fila corresponde a un nodo del flujograma de la sección 4. Las estaciones marcadas como punto crítico tuvieron fricción real durante el proceso — no son riesgos teóricos.

#	Estación	Herramientas	Punto crítico	Sugerencia
1	Ingesta de requisitos	Read, python-docx (bash)	— (sin fricción)	Mantener CLAUDE.md como fuente única de verdad
2	Construcción frontend/backend	Agent (subagente), Write/Edit, node --check	Subagente cortado a mitad, hubo que reanudar	Dividir builds grandes en tareas con checkpoints intermedios
3	Empaquetado PWA	ImageMagick, Write	Bump manual de SW_VERSION, fácil de olvidar	Automatizar el versionado del service worker
4	Diseño responsive	Edit sobre CSS, revisión manual	3 vueltas completas por alcance no acotado; sin navegador headless para verificar	Definir breakpoints/dispositivos objetivo antes de la primera pasada
5	Preparar repo git local	bash (git init/commit) en carpeta montada	El mount no permite borrar/renombrar archivos existentes	Trabajar en una carpeta temporal y copiar el resultado final
6	Acceso a GitHub vía token	— (bloqueado por política)	Confusión por expectativa distinta de otro proyecto/herramienta	Aclarar de entrada qué vía de acceso a GitHub es viable
7	Subida por interfaz web	Finder, github.com/upload	Archivos ocultos (.github, .nojekyll) omitidos sin aviso	Usar "Create new file" con ruta escrita a mano para ocultos
8	Configurar Pages (Actions)	Settings → Pages	Sin el workflow subido, Jekyll publicó el README por error	Verificar visualmente la URL tras cada cambio de Pages
9	git push por terminal	Terminal, git, llavero macOS	403 por credencial de otra cuenta cacheada	Correr "gh auth status" antes de cualquier push
10	Cambio a Deploy from a branch	bash (mv, touch), Edit, GitHub web	Restricción de carpeta (/ o /docs) no conocida de antemano	Decidir el método de deploy antes de fijar la estructura de carpetas
11	Verificación final	Navegador, checklist de flujos	— (resuelto)	Repetir esta verificación tras cualquier cambio futuro

4. Flujograma del proceso

Verde = paso sin fricción. Ámbar = punto crítico encontrado durante el proceso real (no hipotético).



Crítico: varias idas y vueltas — el usuario esperaba la misma facilidad que en otro proyecto con Claude Code (credenciales locales), pero aquí no hay conector OAuth de GitHub disponible ni se aceptan tokens en chat
Sugerencia: aclarar al inicio del proyecto qué vía de acceso a GitHub es viable (web, terminal propio, conector)

7. Subida por la interfaz web de GitHub (drag & drop)

Herramientas: Finder (Cmd+Shift+.) + github.com/upload, más tarde "Create new file" para rutas ocultas
Crítico: el navegador/GitHub filtró silenciosamente los archivos ocultos (.github, .nojekyll) — faltaron sin ningún aviso, y eso alteró el resultado final sin que fuera evidente de inmediato
Sugerencia: usar "Add file → Create new file" con la ruta escrita a mano para cualquier archivo oculto

8. Configurar GitHub Pages: Source = GitHub Actions (1er intento)

Herramientas: Settings → Pages del repo
Crítico: sin el workflow subido (paso 7), GitHub usó su build automático de Jekyll y publicó el README renderizado en vez de la app — sin error visible, solo se detectó al mirar la URL publicada
Sugerencia: verificar visualmente la URL publicada como paso estándar tras cualquier cambio de Pages

9. Intento por terminal: git push con credencial equivocada

Herramientas: Terminal del usuario, git, llavero de macOS (osxkeychain)
Crítico: 403 — el llavero tenía cacheada una credencial de otra cuenta ("Delfos-IQ") sin permiso sobre el repo "pulseiraSAVI"; confusión hasta identificar que eran cuentas distintas
Sugerencia: correr "gh auth status" antes de cualquier push para confirmar la cuenta activa

10. Cambiar a "Deploy from a branch" — renombrar frontend/ a docs/

Herramientas: bash (mv, touch .nojekyll), Edit (README), GitHub web (Create new file + Settings → Pages)
Crítico: "Deploy from a branch" solo admite carpeta / o /docs — esta restricción no se conocía cuando se nombró la carpeta "frontend", obligando a renombrar y re-subir después de ya tener todo publicado
Sugerencia: decidir el método de deploy (rama vs. Actions) antes de fijar la estructura de carpetas

FASE D · Validación

11. Verificación final — demo pública funcionando

Herramientas: navegador (captura del usuario), checklist de flujos (profesional / familia / admin)
Resultado: <https://pulseirasavi.github.io/Pulseira-Savi/> — login real, prompt de instalación PWA visible
Sugerencia: repetir esta verificación visual después de cada cambio futuro de Pages/estructura

5. Sugerencias generales para la próxima iteración

- **Adoptar git/terminal desde el día uno** en vez de la subida web, para un proyecto que seguirá evolyendo — evita las sorpresas de archivos ocultos filtrados y de builds automáticos de Jekyll.
- **Confirmar la cuenta activa de git/GitHub** (`gh auth status`) antes de cualquier push, sobre todo si se trabaja con varias cuentas u organizaciones.
- **Decidir el método de deploy de GitHub Pages** (rama vs. Actions) antes de nombrar la estructura de carpetas del proyecto, ya que "Deploy from a branch" impone la restricción de carpeta / o /docs.
- **Verificación visual como paso estándar** después de cualquier cambio de configuración de Pages, no solo cuando algo parece ir mal.
- **Adelantar la pista legal/organizativa en paralelo** a la técnica: EIPD/DPIA formal, DPAs con Firebase/Cloudflare/Resend, y decidir el responsable del tratamiento — son bloqueantes reales para pacientes piloto y no dependen de ningún avance de código.
- **Cerrar las preguntas abiertas del CLAUDE.md** (destaque visual de alergias/grupo sanguíneo, modelo de verificación clínica única vs. doble, seguridad del acceso por búsqueda, etc.) antes de construir sobre supuestos no confirmados.
- **Probar el flujo completo con un paciente ficticio** contra la infraestructura real (Firebase/Cloudflare reales) antes de plantear cualquier paciente piloto verdadero.